Magadon M1 Submission

A life-simulation game for financial literacy

Problem statement: Vanguard India, *Empowering the Next Generation Through Financial Literacy*

Abstract. Offline-first mobile game in which Class 11–12 students run a fictional character's money, make weekly decisions and see what happens, with the option to rewind and try again. It covers the four pillars of the brief, launches in English and Hindi, and gives partner NGOs a simple dashboard for classes and learning gains. AI helps write content before release, but never runs inside the student app.

1. Problem statement and our understanding

Vanguard India asked for a digital experience that teaches financial literacy to Class 11–12 students from underserved communities in Hyderabad. Students should learn by doing (make choices, see consequences, try again) across four main steps: plan and budget, save and manage, understand investing, and stay financially safe. It should start with about 1,500 students, be able to scale, and educate without recommending investments.

Our understanding of the main constraints:

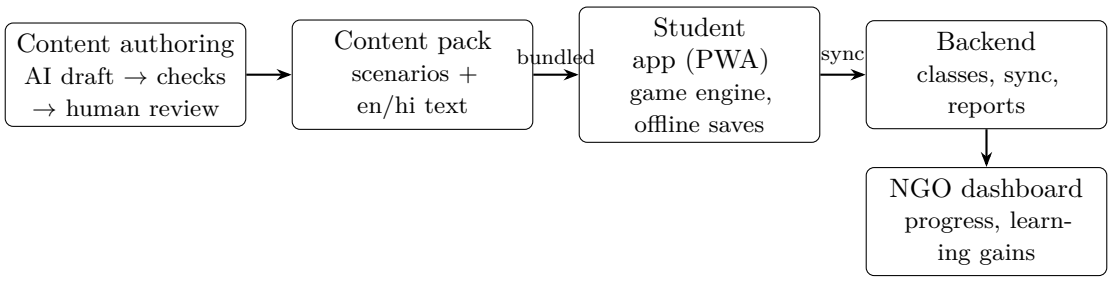
- Lessons must feel like everyday life, not textbook finance, and be in the students' own language.
- Students may have low-end, shared phones and patchy internet.
- Delivery happens through NGOs, who need simple class setup and a view of student progress.
- The users are minors, so privacy and safe, non-advisory content come first.

2. Idea

A student picks a persona, such as Sana, who tutors to save for college fees, or Ravi, and manages that character's money month by month across four jars: Needs, Wants, Goals and Savings. The game opens on a map of everyday places (Home, Shop, School, Hospital). Each week brings a decision based on Hyderabad life, such as a phone recharge, a bus pass or a festival. Every choice shows its consequence and a short lesson, and some consequences arrive weeks later. Students can rewind a few decisions each month, so mistakes become lessons rather than penalties. A month-end report gives a *Money Health Score* based on savings, emergency buffer, goal progress and debt.

Short mini-games cover specific skills: *Scam or Safe?* (spot fraudulent messages) and *Time Machine* (see how savings could grow, always shown as a low/average/high range with the assumptions stated). The app is available in English and Hindi, and the language can be changed at any time. NGO facilitators create a class with a code or QR, follow progress, and compare quiz scores before and after play.

3. Approach and architecture



- **Game engine:** a simulation that always gives the same outcome for the same inputs (seeded, no hidden randomness). This makes rewind, demos and testing reliable.
- **Content as data:** scenarios, lessons and quizzes are data files with translations, so content can change without code changes and a new language needs only one new text file.

- **Offline-first:** the game works without a connection, saves on the phone, and syncs progress when a connection is available.
- **AI only at authoring time:** AI helps draft content and Hindi translations, which are checked automatically and reviewed by a person. No AI runs inside the student app, so it works offline and no student data is sent anywhere.
- **Guardrails and privacy:** only fictional financial products; projections shown as ranges with assumptions; students join with a nickname and class code only; facilitators see only their own classes; no ads or trackers. Parental consent under India's DPDP Act is a rollout question for Vanguard and the NGO partners.

The system has three parts: a student app that runs the game on the phone, a small backend for joining classes, syncing progress and building reports, and a web dashboard for NGO staff. Content is prepared ahead of time and shipped with the app. The planned stack is TypeScript throughout, a progressive web app for students, and a Supabase (PostgreSQL) backend hosted in India (Mumbai), with the apps on Vercel; details may change as we build.

4. Planned testing methodology

- **Engine unit tests:** same inputs give the same game, the input state is never changed, rewind restores the earlier state, jar balances never go negative, delayed consequences arrive on time (including across months), and saves load back exactly.
- **Simulation runs:** play many games automatically to check balance and catch errors.
- **Content checks:** an automated linter checks that content matches the schema, every text exists in English and Hindi, and nothing names real products or promises returns. A person then reviews the content, and a Hindi speaker checks the most-used cards.
- **Backend tests:** joining a class, syncing the same progress twice without duplicates, and staff seeing only their own classes.
- **Device and offline tests:** play on a low-end Android phone and on a throttled connection, including in airplane mode, then reconnect and check that progress syncs.
- **User testing:** first-time users try the app without help (target: first decision in under 3 minutes), and learning is measured with quizzes before and after play.

All automated checks run before every commit and in continuous integration.